

SRS

Software Requirements Specification (SRS)

Project: Endur (Performance Review & Feedback Management System)

Team: 39_endur

1. Preface

1.1 Expected Readership

This document is intended for:

- **Project Developers (Team 39_endur):** To understand the technical specifications and constraints.
 - **Domain Expert (Dean of Academics):** To validate that the operational workflows match university policy.
 - **Project Evaluators:** To assess the depth of requirement analysis and system modeling.
-

2. Introduction

2.1 Need for the System

The current end-semester feedback model fails on two critical fronts:

Pain Point 1: Timing Latency

Feedback arrives too late to benefit the current batch and is often skewed by "Revenge Ratings" submitted after grades are released.

Pain Point 2: Engagement Fatigue

The Dean reports that **only ~30% of students participate** because the process is viewed as a "chore" rather than a meaningful contribution.

2.2 System Functions

- **Continuous Feedback Cycles:** Allows multiple review windows per semester.

- **Role-Based Dashboards:** Distinct views for Student (Submission), Faculty (Reflection), and HOD (Oversight).
 - **Gap Analysis:** Automated comparison between Faculty Self-Reflection and Student Perception.
 - **Anonymity Preservation:** Decoupling of student identity from feedback data.
-

3. Glossary

Refer to [definitions.yaml](#) in the repository for the complete Ubiquitous Language.

- **FeedbackCycle:** A defined, time-bound period for data collection.
 - **GapAnalysis:** The quantitative difference between a professor's self-rating and the students' average rating.
 - **AnonymizedReport:** A generated view where student identities are stripped to ensure psychological safety.
 - **ActionReport:** A formal plan submitted by faculty to the HOD to address performance gaps.
 - **ReviewOfReviews:** A meta-evaluation process accessible to both students and faculty members that assesses the effectiveness, fairness, and usefulness of the feedback system itself, including question design and timing.
 - **FeedbackTrend:** A longitudinal view of PerformanceScores across multiple FeedbackCycles, used to evaluate improvement, stagnation, or decline over time.
 - **SelfReflection:** A reflective input provided to a FacultyMember and Student to assess their own performance, challenges, and improvements during a semester.
 - **ReviewCheckIn:** A scheduled, lightweight review interaction between a FacultyMember and the DepartmentHead (or academic reviewer) focused on discussing feedback trends, gap analysis, and progress on previously committed action items, rather than re-evaluating raw scores.
 - **FeedbackResponse:** A single, immutable submission made by a Student during a FeedbackCycle for a specific CourseOffering, consisting of PerformanceScores mapped to EvaluationParameters. Each FeedbackResponse represents one complete feedback instance and is used as an atomic unit for aggregation, analysis, and auditing.
-

4. User Requirements Definition

High-level services provided for the user, described in natural language.

4.1 Student Services

- **UR-01:** Students shall be able to submit weekly/monthly [FeedbackCycle](#) which are lightweight (taking < 30 seconds) to ensure high participation.
- **UR-02:** Students shall be able to view a history of their past submissions for transparency.
- **UR-03:** Students should be able to fill monthly feedback form on [ReviewOfReviews](#).

4.2 Faculty Services

- **UR-04:** Faculty shall be able to submit a [SelfReflection](#) assessment before viewing student feedback.
- **UR-05:** Faculty shall be able to view their performance and [FeedbackTrends](#).
- **UR-06:** Faculty shall be able to submit an [ActionReport](#) if their performance scores trigger a system alert.

4.3 Administrative (HOD) Services

- **UR-07:** The Department Head (HOD) shall be able to view aggregated reports for all faculty members in their departments.
- **UR-08:** The HOD shall be able to conduct [ReviewCheckIn](#) with the concerned faculty members.
- **UR-09:** The HOD is able to define **EvaluationParameter** for every [FeedbackCycle](#).

4.4 Dean Services

- **UR-10:** The Dean shall be able to view institution-wide performance reports covering all departments, faculty members, and courses.
- **UR-11:** The Dean shall be able to act as the approving authority when a conflict of interest is identified, such as when a Department Head is reviewing their own performance.
- **UR-12:** The Dean shall be able to view [ReviewCheckIn](#) records and [ActionReports](#) from any department for oversight purposes.
- **UR-13:** The Dean shall be able to track long-term [FeedbackTrends](#) across academic years to identify overall strengths, risks, and areas needing improvement at the institutional level.
- **UR-14:** The Dean shall be able to review [ComplianceAudit](#) logs and flagged feedback submissions to ensure fairness, integrity, and adherence to academic policies.
- **UR-15:** The Dean shall be able to freeze, reopen, or invalidate [FeedbackCycles](#) in exceptional academic or administrative situations, with all such actions recorded in the audit log.

5. System Requirements Specification

5.1 Functional Requirements (FR)

FR-01: [FeedbackCycle](#) Window Enforcement

The system shall enforce FeedbackCycle submission boundaries based on configured start and end timestamps.

- Feedback submission requests received before `FeedbackCycle.startTimestamp` or after `FeedbackCycle.endTimestamp` shall be rejected.
- Rejected submissions shall return a validation error indicating that the FeedbackCycle is inactive.
- Enforcement shall apply uniformly across all FeedbackCycle types.

FR-02: [Attendance-Weighted PerformanceScore](#) Calculation

The system shall calculate PerformanceScore using AttendanceWeightedFeedback when attendance data is available for a [CourseOffering](#).

- If student attendance exceeds 75%, the applied weight shall be 1.0.
- Attendance thresholds and corresponding weights shall be configurable.
- If attendance data is unavailable, unweighted averaging shall be applied.
- Weighting logic shall be applied per EvaluationParameter.

FR-03: [GapAnalysis](#) Computation

The system shall compute the gap using the following logic:

$$Gap = |SelfReflection.Score - Average(StudentPerformanceScore)|$$

FR-04: [ActionReport](#) Immutability

The system shall enforce immutability of ActionReports after submission.

- Once submitted, an ActionReport shall become read-only to the [FacultyMember](#).
- Editing, deletion, or overwriting of a submitted ActionReport shall not be permitted.
- A new ActionReport may only be submitted in a subsequent [FeedbackCycle](#).
- [DepartmentHead](#) access to ActionReports shall be read-only.

FR-05: [AnonymizedReport](#) Generation

The system shall generate AnonymizedReports for each FacultyMember and CourseOffering after the closure of a FeedbackCycle.

- AnonymizedReports shall aggregate PerformanceScores per EvaluationParameter.
- Individual student identities and raw submissions shall not be exposed.

- QualitativeFeedback shall be grouped and anonymized before inclusion.
- AnonymizedReports shall be immutable once generated.

FR-06: [FeedbackTrend](#) Computation

The system shall compute FeedbackTrends across multiple FeedbackCycles for the same CourseOffering and FacultyMember.

- FeedbackTrends shall be based on aggregated PerformanceScores.
- Trends shall be calculated per EvaluationParameter.
- Historical FeedbackTrends shall not be recalculated when new cycles are added.
- FeedbackTrends shall not alter historical data.

FR-07: [ComplianceAudit](#) for Feedback Integrity

The system shall perform ComplianceAudit checks on feedback submissions to detect integrity violations.

- The system shall flag **RandomTicking** patterns, including:
 - Identical PerformanceScores across all EvaluationParameters.
 - Multiple submissions within an unusually short time window.
- Flagged submissions shall be excluded from PerformanceScore calculations.
- ComplianceAudit results shall be logged for administrative review.

FR-08: [ReviewCheckIn](#) Record Management

The system shall store ReviewCheckIn records associated with a FacultyMember and CourseOffering.

- ReviewCheckIn records shall reference:
 - Relevant FeedbackTrends
 - Associated GapAnalysis results
 - Related ActionReports
- ReviewCheckIn records shall be immutable once saved.

FR-09: Institutional Performance Aggregation

The system shall generate institution-wide analytics for the [Dean](#) by aggregating data across all Departments.

- The system shall compute an overall **PerformanceScore** for the entire institution by averaging aggregated scores from all AnonymizedReports.
- The system shall allow the Dean to filter **FeedbackTrends** by:
 - Department
 - Academic Year
 - CourseContentQuality
- The system shall generate a comparative view identifying the **highest and lowest performing Departments** based on aggregated EvaluationParameters.

FR-10: Administrative Conflict Resolution Workflow

The system shall enforce a specific workflow when a DepartmentHead is the subject of a review.

- If the FacultyMember associated with a CourseOffering is also the current DepartmentHead, the system shall automatically reassign **ReviewCheckIn approval authority** to the Dean.
- The system shall prevent the DepartmentHead from viewing or interacting with their own **ActionReport** as a reviewer.

FR-11: Emergency Cycle Management

The system shall allow the Dean to manually override the state of a FeedbackCycle in exceptional circumstances.

- The Dean shall be able to **Freeze** an active FeedbackCycle, immediately rejecting new [FeedbackResponse](#) submissions regardless of the configured end timestamp.
- Any manual state change triggered by the Dean shall automatically generate a **ComplianceAudit log entry** requiring a justification note.

FR-12: Long-Term Trend Analysis

The system shall compute multi-year FeedbackTrends to support institutional strategic assessment by the Dean.

- The system shall aggregate **PerformanceScores** across multiple academic years to visualize long-term progression.
- The system shall highlight trends in **CourseContentQuality** independently from delivery-related EvaluationParameters.
- Historical trend data shall remain immutable once calculated.

5.2 Non-Functional Requirements (NFR)

- **NFR-01 (Anonymity & Privacy):**

The system shall ensure that `Student_ID` is never retrievable, inferable, or reconstructible from any `FeedbackResponse`, `AnonymizedReport`, or downstream analytics store.

- **NFR-02 (Scalability):**

The system shall support a minimum of *500 concurrent write operations* within the final *10 minutes of an active FeedbackCycle* without data loss or degradation of response correctness.

- **NFR-03 (Availability):**

The system shall maintain *99.9% service availability* during institution-defined peak periods, including active exam weeks and `FeedbackCycle` submission windows.

- **NFR-04 (Immutability & Data Integrity):**

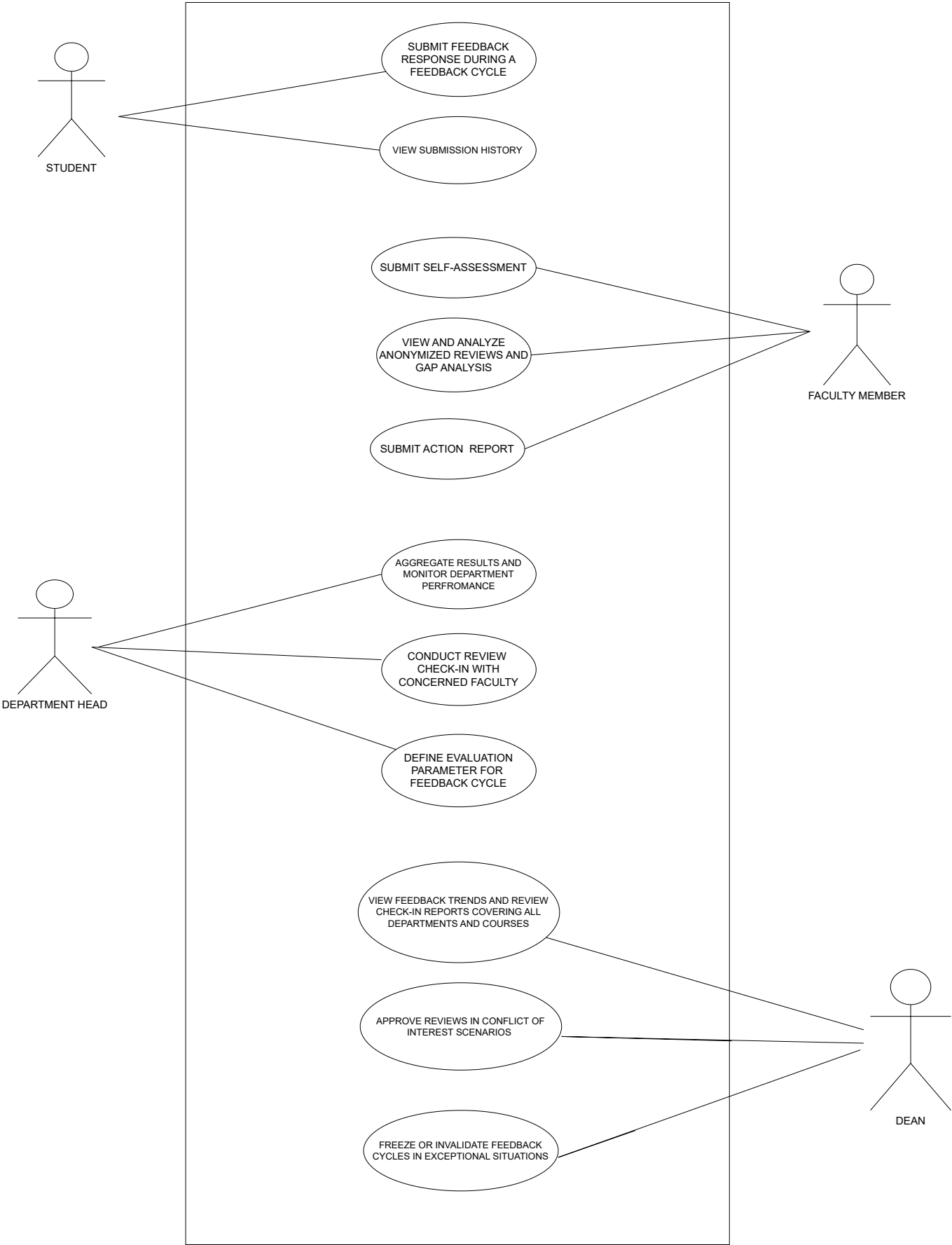
The system shall enforce immutability for all finalized artifacts, including submitted feedback responses, generated `AnonymizedReports`, `ActionReports`, and `ReviewCheckIn` records.

- **NFR-05 (Auditability):**

The system shall maintain an append-only audit log capturing all create and state-transition events.

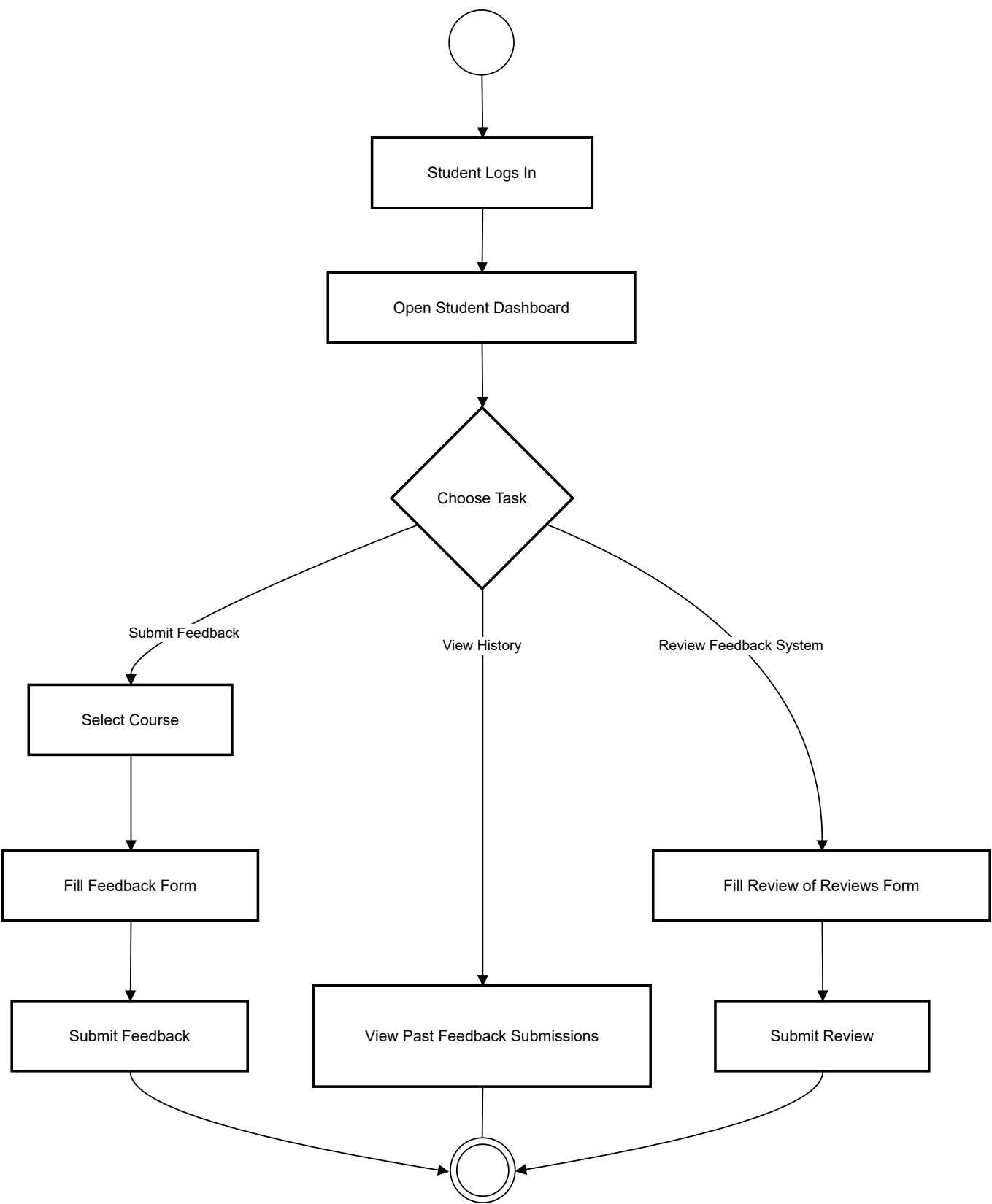
6. System Models

6.1 Use Case Diagram

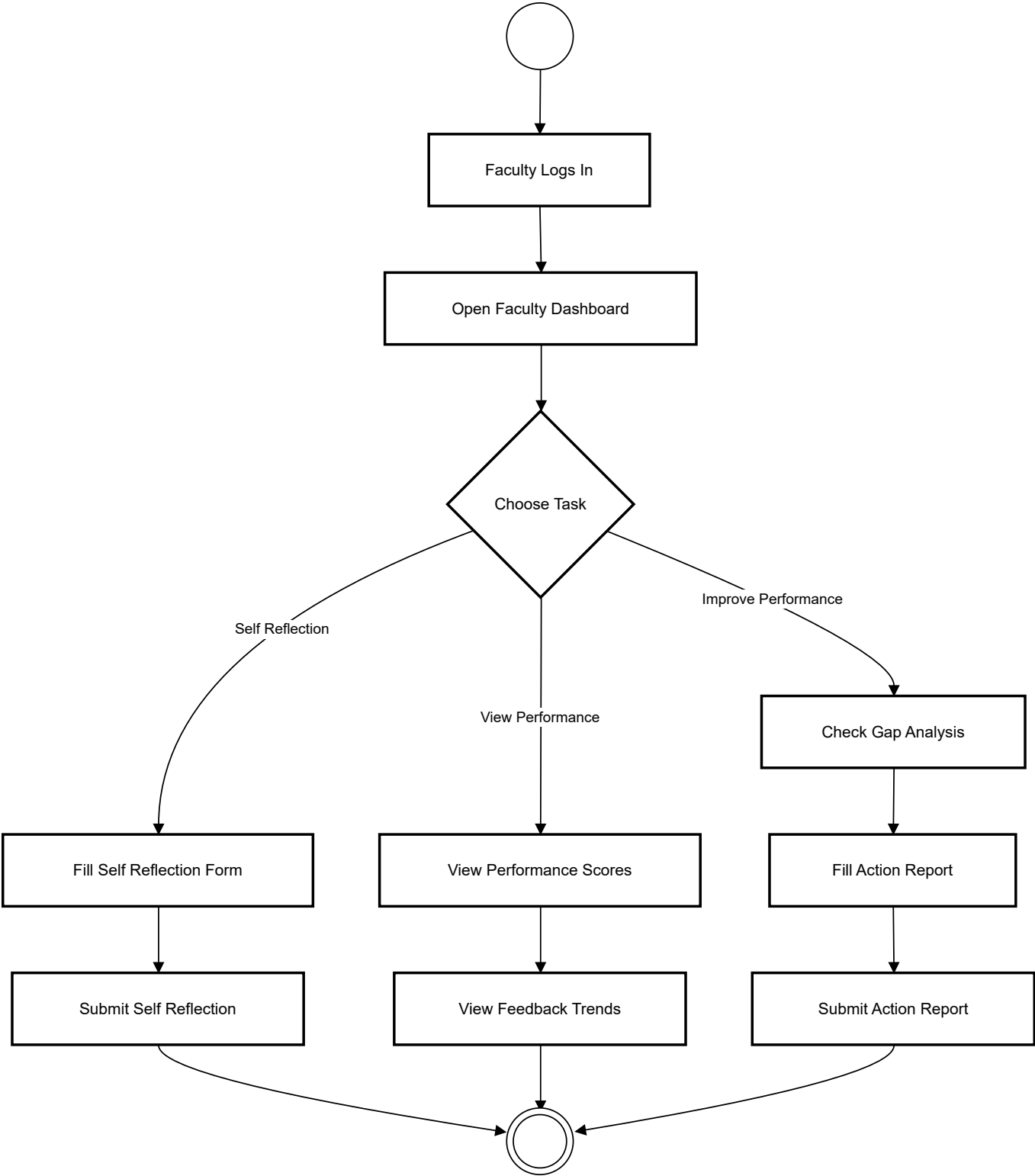


6.2 Activity Diagrams

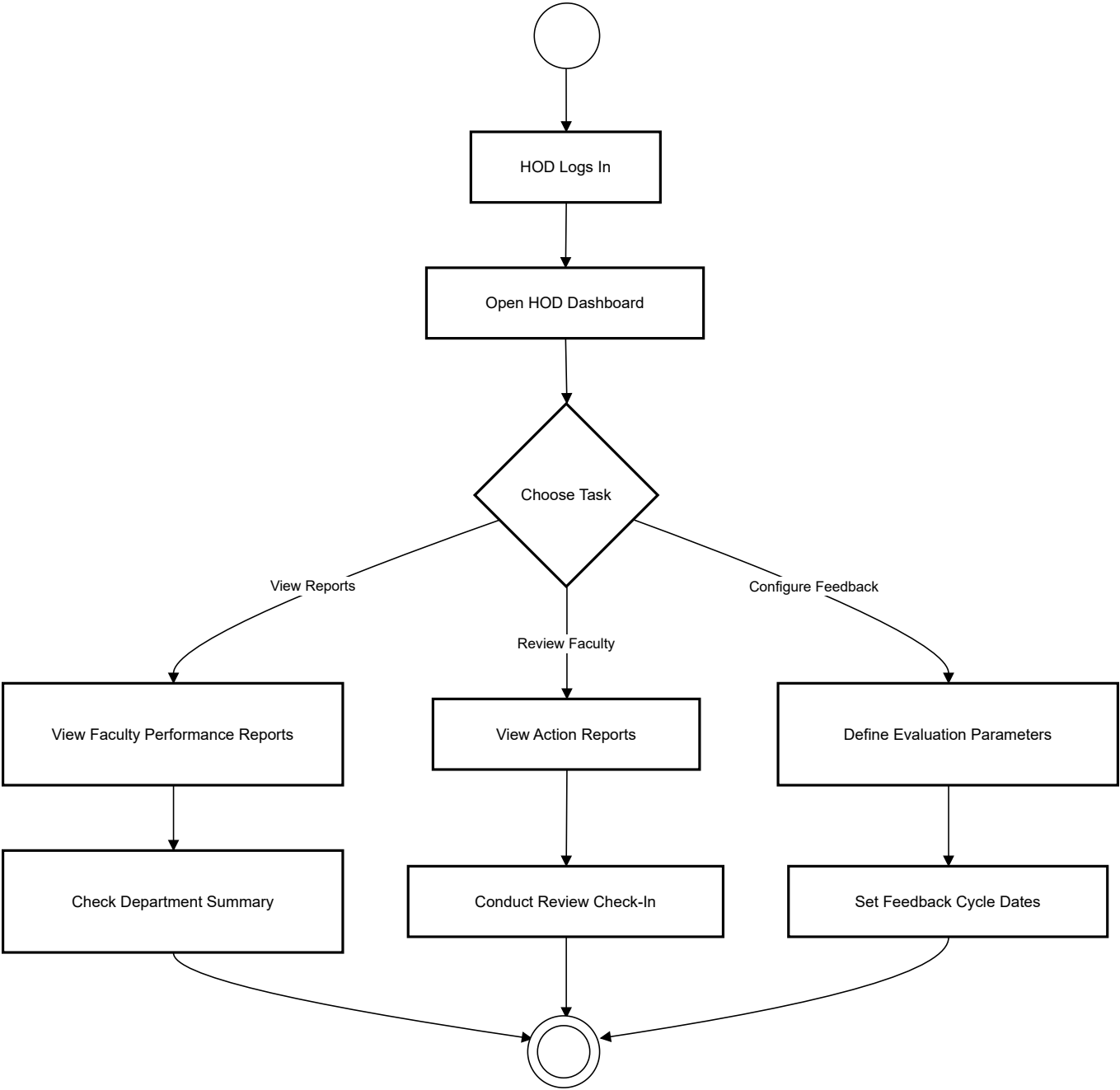
1) Student Activity Diagram



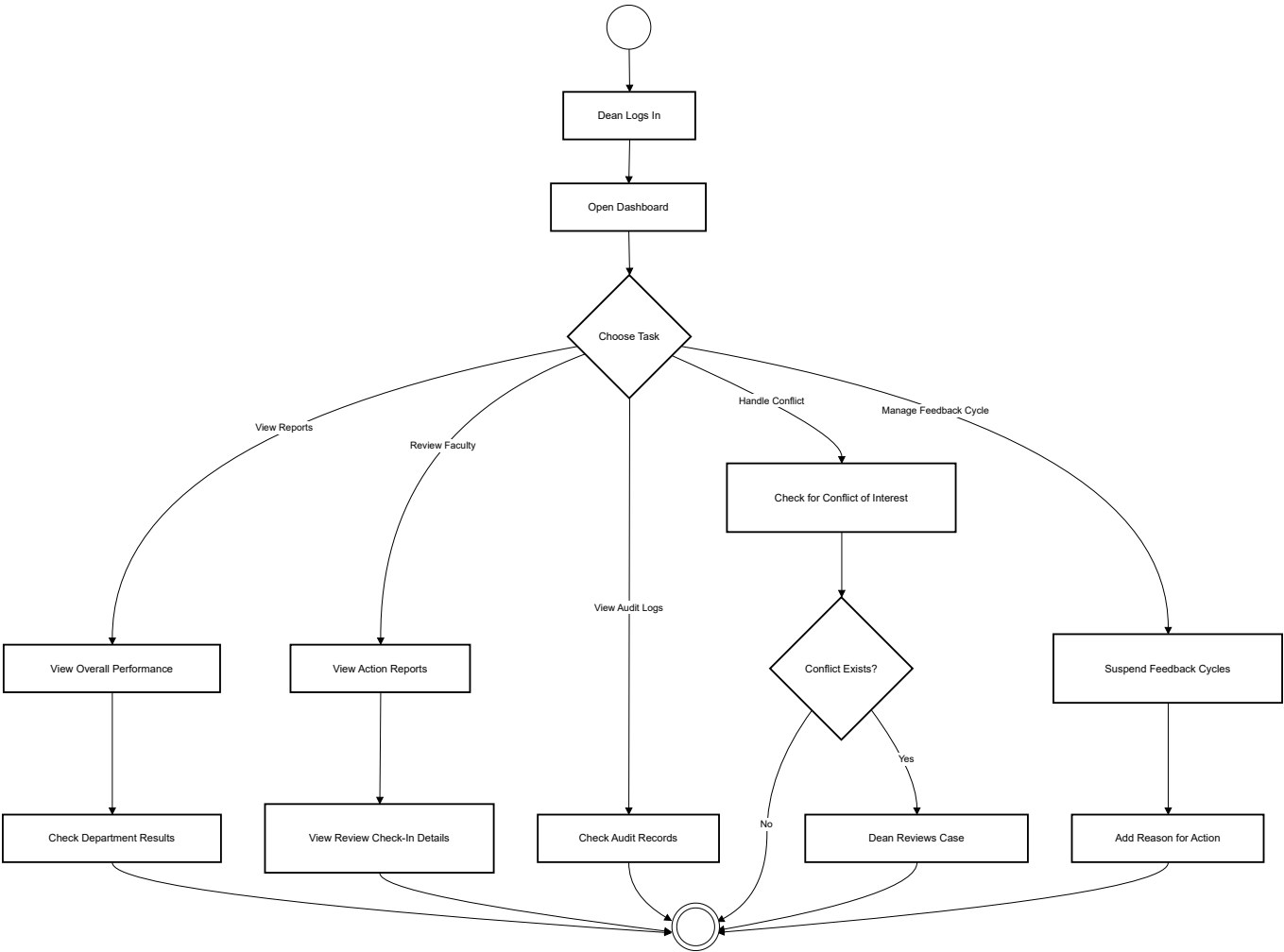
2) Faculty Member Activity Diagram



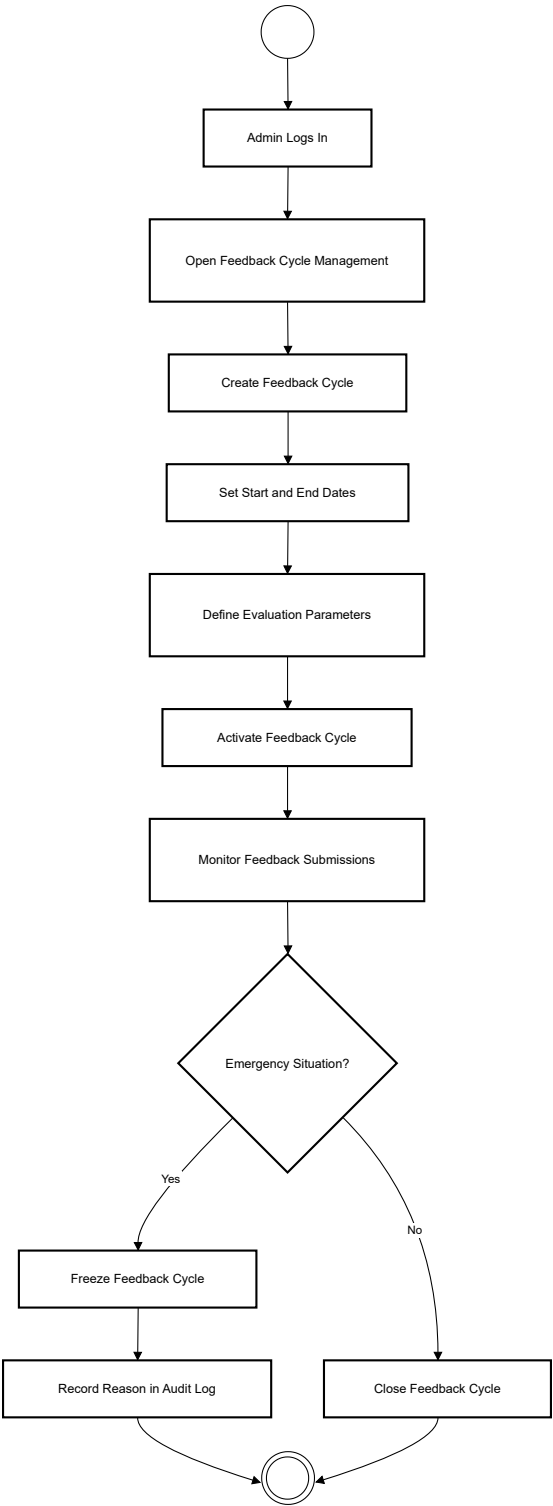
3) HOD Activity Diagram



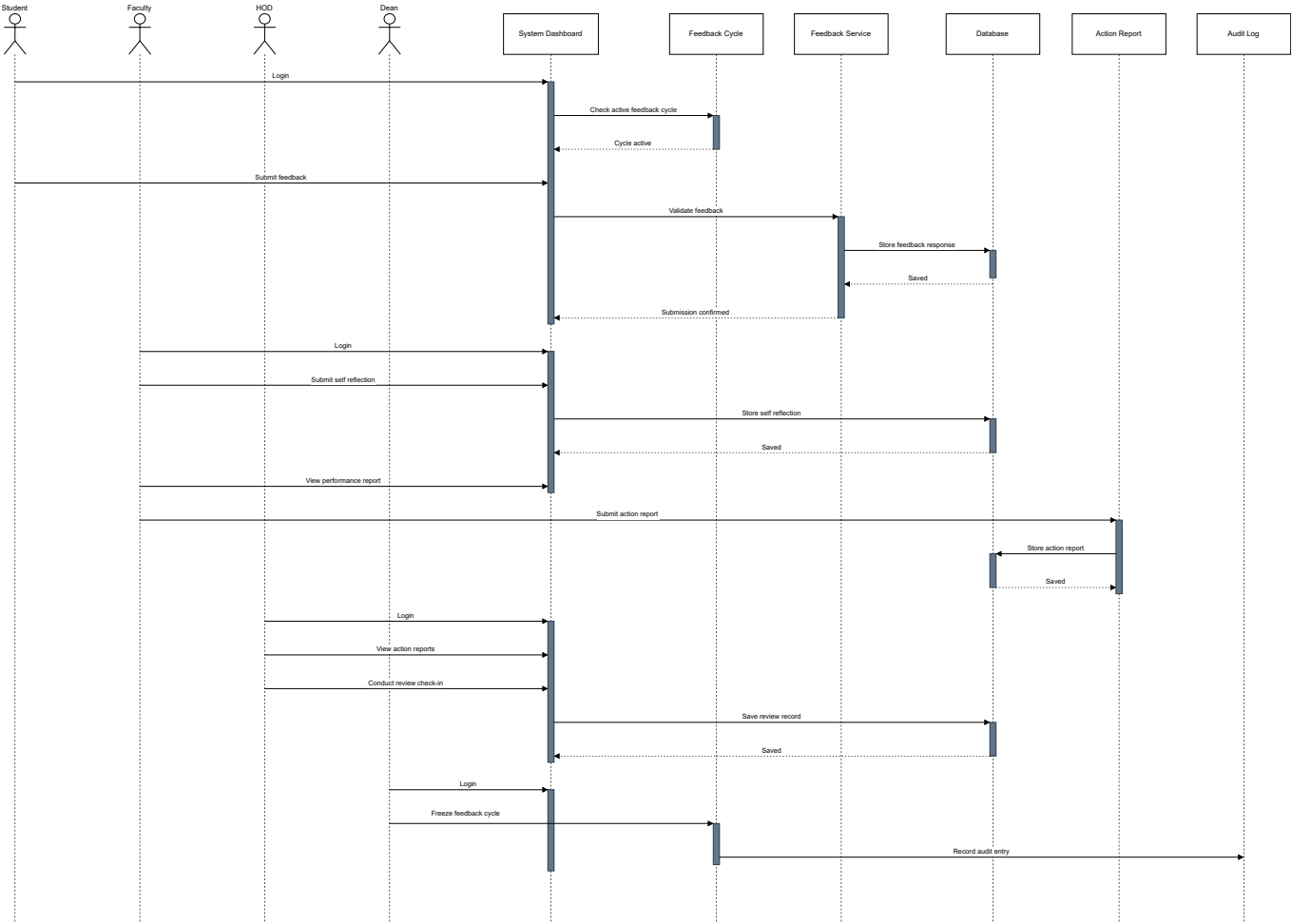
4) Dean Activity Diagram



5) Admin Activity Diagram



6.3 Sequence Diagram



7. Index

A

- [ActionReport](#)
- [AnonymizedReport](#)
- [AttendanceWeightedFeedback](#)

C

- [ComplianceAudit](#)
- [CourseOffering](#)

D

- [DepartmentHead](#)
- [Dean](#)

F

- [FacultyMember](#)
- [FeedbackCycle](#)
- [FeedbackResponse](#)
- [FeedbackTrend](#)

G

- [GapAnalysis](#)

R

- [ReviewCheckIn](#)
- [ReviewOfReviews](#)